

TP5 – Observateurs et Sujets

IUT de Paris Descartes – R201 – Base de la programmation objet

Denis Poitrenaud et Mikal Ziane

Objectifs

- Mettre en œuvre une relation entre observateurs et sujets observés.
- Comprendre les bases d'un mécanisme de notification.

Une usine de voitures – suite

Nous reprenons les sources du **sujet** du TP de la semaine précédente et non **pas** de son corrigé. Il n'est plus question cette semaine de modifier les freins ni le moteur. Il nous est demandé de simuler le compteur de vitesse équipant les voitures de la marque.

Le rôle du compteur est de maintenir à jour l'affichage de la vitesse courante. Il est plus simple que le compteur n'ait pas à mettre à jour l'affichage à intervalle régulier. La voiture indiquera donc au compteur quand l'affichage doit être mis à jour. Elle le fera chaque fois que la valeur de la vitesse est modifiée.

Un *design pattern*, c'est à dire une solution bien connue qui a reçu un nom, *Observateur*, répond à ce problème. Ce pattern est utilisé dans pratiquement toutes les interfaces homme machine modernes. Une implémentation rudimentaire de ce pattern est incluse dans Java mais elle est dépréciée (*deprecated*) depuis longtemps. La solution préconisée de nos jours repose sur une classe que nous appellerons ici **Changement**. Un changement est événement dans lequel une propriété d'un certain nom ("vitesse" pour nous) a changé dans un sujet (une voiture pour nous).

Un **sujet** est un objet qui permet à des **observateurs** de s'abonner aux changements d'une **propriété** du sujet afin d'être prévenu quand la valeur de cette propriété change. Lorsqu'un changement de valeur a lieu sur une propriété d'un sujet, ce dernier prévient tous les observateurs inscrits pour "écouter" les changements de cette propriété.

Exercice 1 Définissez un paquetage **observation** et une interface **observation.ISujet**. Prévoyez une méthode pour abonner un **observation.IObservateur** aux changements de valeur d'une propriété dont le nom (une **String**) est fourni. Il faut aussi pouvoir résilier un abonnement. Enfin, une méthode doit retourner l'ensemble des propriétés (des **String**) auxquelles on peut s'abonner.

Exercice 2 Définissez un **record observation.Changement** qui mémorise le sujet concerné, le nom de la propriété et sa nouvelle valeur (convertie en **String**). Notez que comme la valeur est fournie cela évite à l'observateur de savoir quelle méthode appelée pour récupérer cette valeur d'autant que l'observateur ne connaît pas forcément la classe du sujet ! Notez aussi que comme la valeur est convertie en **String** ce qui évite d'avoir à préciser son type.

Définissez une interface `observation.IObservateur` avec une unique méthode, `miseAJour` qui sera appelée par un sujet pour prévenir un observateur qu'un changement a eu lieu sur une propriété qu'il "écoute".

Exercice 3 Définissez une classe `observation.Gestionnaire`. Chaque objet de cette classe gèrera les observateurs d'un sujet en fournissant les méthodes suivantes.

- `void abonner(String propriété, IObservateur observateur)` qui abonne un observateur aux changements concernant une propriété du sujet.
- `void résilier(String propriété, IObservateur observateur)` qui résiliera l'abonnement à une propriété du sujet.
- `void résilier(IObservateur observateur)` qui résiliera l'abonnement à toutes les propriétés du sujet.
- `void notifierChangement( String propriété)` qui appellera la méthode `mettreAJour` des abonnés à une propriété lorsque sa valeur a changé.

Quand une propriété est mentionnée dans une de ces méthodes, vous vérifierez qu'il s'agit bien de "vitesse" sinon une `IllegalArgumentException` sera levée. Si un observateur n'est pas abonné à une propriété aucune exception ne sera levée s'il résilie. De même si un observateur s'abonne plusieurs fois à la même propriété aucune exception ne sera levée.

Exercice 4 Programmez une classe `ihm.CompteurVitesse` représentant un compteur de vitesse. Il affichera le nom de la voiture suivi de sa vitesse courante sur `System.out`. Cette classe implémentera l'interface `IObservateur`.

Modifiez la classe `Voiture` pour qu'elle implémente l'interface `ISujet`. Pour cela elle inclura un objet de type `Gestionnaire` à qui elle déléguera les méthodes de l'interface.

Exercice 5 Modifier la classe `Appli` pour que l'affichage de la vitesse des voitures utilise un compteur de vitesse.